CSL7360

# Computer Vision

## Lecture 33: Graph Neural Networks

**Dr. Hardik Jain**
Assistant Professor
Department of Computer Science and Engineering
IIT Jodhpur

Reference: Sanchez-Lengeling, et al., "A Gentle Introduction to Graph Neural Networks", Distill, 2021.

1

## What are Graphs

A set of objects, and the connections between them



2

## Graph Representation

- A graph represents the relations (*edges*) between a collection of entities (*nodes*)
- **V:** Vertex (or node) attributes
  - e.g., node identity, number of neighbours
- **E:** Edge (or link) attributes and directions
  - e.g., edge identity, edge weight



3

## Where are Graphs?

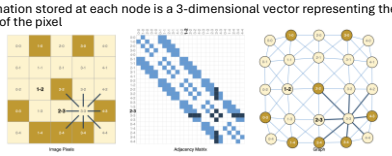
Wherever we have objects and connections between them



4

## Images as graphs

- Images can be represented as graphs with regular structure, where
  - Each pixel represents a node
  - Is connected via an edge to adjacent pixels
  - Each non-border pixel has exactly 8 neighbors
  - Information stored at each node is a 3-dimensional vector representing the RGB value of the pixel






5

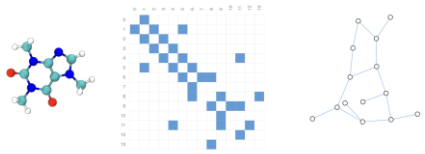
## Text as graphs

- Text can be digitized by associating indices to each character, word, or token,
- Then representing text as a sequence of these indices.
- This creates a simple directed graph, where each character or index is a node and is connected via an edge to the node that follows it.

6

## Molecules as graphs

- Different pairs of atoms and bonds have different distances (e.g. single-bonds, double-bonds).
- It's a very convenient and common abstraction to describe this 3D object as a graph, where nodes are atoms and edges are covalent bonds.



(Left) 3d representation of the Caffeine molecule (Center) Adjacency matrix of the bonds in the molecule (Right) Graph representation of the molecule.



7

## Social Networks as graphs

- Social networks are tools to study patterns in collective behaviour of people, institutions and organizations
- A graph can be built that represents group of people by modelling individuals as nodes, and their relationships as edges

(Left) Image of a scene from the play "Othello". (Center) Adjacency matrix of the interaction between characters in the play. (Right) Graph representation of these interactions.



8

## Other Examples

- **Citations**: where each paper is a node, and each *directed* edge is a citation between one paper and another
- **Scene**: want to tag objects in visual scenes. We can then build graphs by treating these objects as nodes, and their relationships as edges
- Machine learning models, programming code and math equations: can also be phrased as graphs, where the variables are nodes, and edges are operations that have these variables as input and output. You might see the term "dataflow graph" used in some of these contexts.



9

# Tasks on Graphs

Graph-level, Node-level, and Edge-level

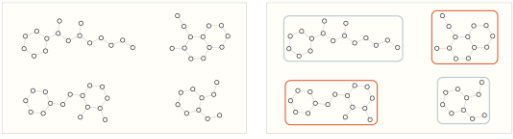


10

## Graph-level

Tasks

- **Goal**: To predict the property of an entire graph
- Eg: For a molecule represented as a graph, we might want to predict what the molecule smells like, or whether it will bind to a receptor implicated in a disease.



**Input:** graphs **Output:** labels for each graph, (e.g., "does the graph contain two rings?")



11

## Node-level

Tasks on Graphs

- Node-level tasks are concerned with predicting the identity or role of each node within a graph
- **A classic example is Zach's karate club**: The dataset is a single social network graph made up of individuals that have sworn allegiance to one of two karate clubs after a political rift. As the story goes, a feud between Mr. Hi (Instructor) and John H (Administrator) creates a schism in the karate club. The nodes represent individual karate practitioners, and the edges represent interactions between these members outside of karate. The prediction problem is to classify whether a given member becomes loyal to either Mr. Hi or John H, after the feud. In this case, distance between a node to either the Instructor or Administrator is highly correlated to this label.

**Input:** graph with unlabled nodes **Output:** graph node labels

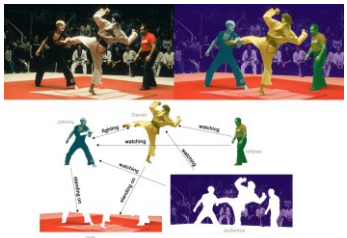


12

## Edge-level

Tasks on Graph

- **Scene Understanding**: Beyond identifying objects in an image, deep learning models can be used to predict the relationship between them.
- Given nodes that represent the objects in the image, we would want to predict which of these nodes share an edge or what the value of that edge is.
- To discover connections between entities, a fully connected graph is considered and then based on the predicted value, prune edges to arrive at a sparse graph.






13

# Learning on Graphs

Challenges and Solution

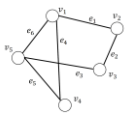


14

## Graph Representation

Challenges

- A graph can be denoted as G = $\{V, E\}$
  - where $V = \{v_1, \ldots, v_N\}$ is a set of $N = |V|$ nodes
  - $E = \{e_1, \ldots, e_M\}$ is a set of M edges




- Issues with Graph:
  - How to represent graphs to be compatible with neural networks.
  - Machine learning models typically take rectangular or grid-like arrays as input.
  - So, it's not immediately intuitive how to represent them in a format that is compatible with deep learning.



15

## Information Representation

Challenges

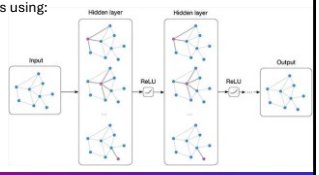
- Graphs have three types of information that can be potentially want to use to make predictions:
  - Nodes
  - Edges
  - Connectivity
- **Straightforward**: Nodes and edges. For e.g., with nodes we can form a node feature matrix $N$ by assigning each node an index $i$ and storing the feature for node $i$ in $N$. While these matrices have a variable number of examples, they can be processed without any special techniques.
- **Complicated**: Representing graph's connectivity. Perhaps the most obvious choice would be to use an adjacency matrix, since this is easily tensorisable. However, this representation has a few drawbacks.
  - Number of nodes in a graph can be on the order of millions, and the number of edges per node can be highly variable. Often, this leads to very sparse adjacency matrices, which are space-inefficient.
  - Various adjacency matrices can encode the same connectivity, and there is no guarantee that these different matrices would produce the same result in a deep neural network (that is to say, they are not permutation invariant).



16

## Graph Convolutional Network (GCN)

- GCNs are tailored to work with non-Euclidean data
  - Each node represents an entity
    - Vector representations: Graph's structural and feature information
  - Edges represent the relationships between these entities
  - Goal is to learn node embeddings using:
    - features of a node
    - Its neighborhood




Deep Learning for Graphs



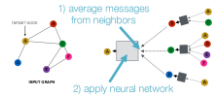
17

## Architecture of GCN

Graph Convolutional Network

**Convolutional Layers**

- Each layer updates the feature representation of a node by aggregating the features of its neighbors




**Pooling Layers**

- Reduces the dimensionality of the graph by merging nodes, which helps in capturing hierarchical structures

18

## Node Features using EdgeConv

- Concatenate the node features, pass the concatenated feature through a MLP to get edge features
- A symmetric aggregation function is applied upon the edge features to get new node feature

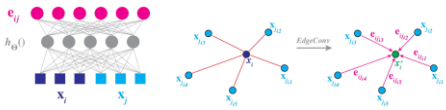



Fig. 2. **Left**: Computing an edge feature, $\mathbf{e}_{ij}$ (top), from a point pair, $\mathbf{x}_i$ and $\mathbf{x}_j$ (bottom). In this example, $h_\Theta()$ is instantiated using a fully connected layer, and the learnable parameters are its associated weights. **Right**: The EdgeConv operation. The output of EdgeConv is calculated by aggregating the edge features associated with all the edges emanating from each connected vertex.

Wang et al., "**Dynamic Graph CNN for Learning on Point Clouds**", *Transactions on Graphics, 2019*



21

## EdgeConv

- Extracts local information for every node
- Beyond the coordinate space, it works in the feature space so that it can draw connection between semantically close nodes
- Permutation invariant and easy to implement because it only involves k-NN and symmetric operation
- By simply plug in EdgeConvs into a network, we get Dynamic Graph CNN (DGCNN).
- Dynamic Graph!
  - The authors find it beneficial to recompute the graph using the nearest neighbors in the feature space produced by each layer.
  - A new graph, instead of a static graph for each layer!
  - A major difference from previous graph CNNs.

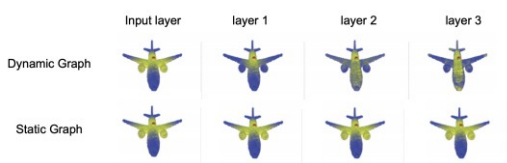
Wang et al., "Dynamic Graph CNN for Learning on Point Clouds", Transactions on Graphics, 2019



22

## Why Dynamic Graph?

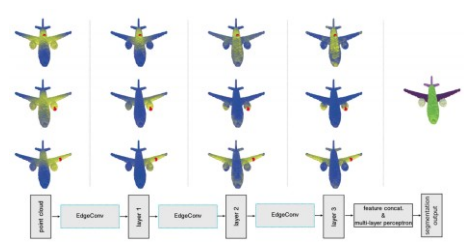
- In later stages, DGCNN learns to gather semantically similar points, despite the large distance in the original coordinate space.






23

## Learning in Layers






24

## Limitations

- GPU Memory Consumption
  - Construct the graph requires to compute all pairwise distances between all the points
  - Very high GPU memory consumption
- Varying Density
  - Point clouds can have varying density at different positions; a fixed k may be not flexible enough
- Large-scale Point Cloud
  - DGCNN does not compress the number of points during forward pass



25

# Key Advancements

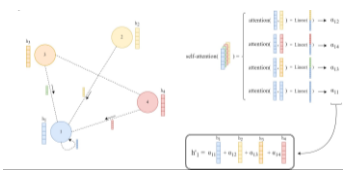
Architectural Improvements



26

## Graph Attention Networks

- Introduces an attention mechanism into the message-passing process, allowing the model to learn and assign different importance weights to neighboring nodes (e.g., highly relevant objects get a stronger connection/message).



Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., & Bengio, Y. (2017). Graph attention networks. arXiv preprint arXiv:1710.10903.



27

## Scalability Techniques

- **Challenge**: Standard GNNs require loading the entire graph structure and features into memory to compute layer-wise aggregation, which is infeasible for graphs with billions of nodes.
- **Cluster-GCN Solution**: This method first uses a graph clustering algorithm to partition the nodes into small, manageable groups (clusters). In each training iteration (mini-batch), only the nodes within a few sampled clusters are processed, and the GNN convolution is performed only over the edges within those clusters (or slightly expanded to neighbors).
- **Advantage**: This drastically reduces memory usage and computation cost, enabling the training of GNNs on industrial-scale datasets.

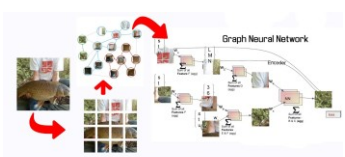
Chiang, W. L., Liu, X., Si, S., Li, Y., Bengio, S., & Hsieh, C. J. (2019, July). Cluster-GCN: An efficient algorithm for training deep and large graph convolutional networks. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining* (pp. 257-266).



28

## Vision GNN (ViG) Architectures

- **Node Creation**: Image is initially split into a number of patches, which serve as the nodes in the graph.
- **Edge Construction**: Edges are constructed by connecting nearest neighbors or patches with high feature/semantic similarity. This can be done dynamically.
- **Processing**: Uses specialized Graph Convolution modules to aggregate and update information among these nodes (patches).




Vision GNN: An Image is Worth Graph of Nodes

Captures **irregular and complex object shapes** and global relationships more flexibly than standard CNNs.

Han, K., Wang, Y., Guo, J., Tang, Y., & Wu, E. (2022). Vision gnn: An image is worth graph of nodes. Advances in neural information processing systems, 35, 8291-8303.



29

## Hybrid & Modular Architectures

**CNN + GNN**

- CNN: Used as a backbone to extract powerful local features and regional proposals (e.g., from object detection).
- A subsequent GNN layer models the relationships (edges) between these proposals/regions

**Transformer + GNN**

- GNNs are used to enhance the feature tokens generated by Vision Transformers, providing an explicit mechanism to model the semantic dependencies among visual tokens.

**Best of Both Worlds**

CNNs/Transformers handle efficient local feature extraction, and GNNs excel at **global relational reasoning** and context understanding



30

# Thank you



31